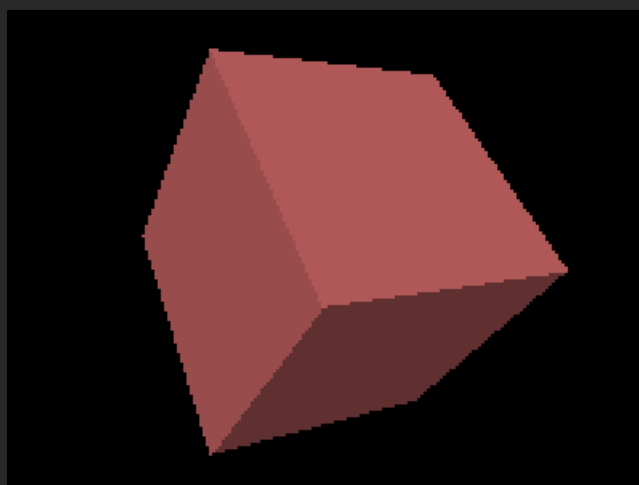


Inigo Quilez :: articles :: rendering a cube in qbasic - 2003



I never did anything impressive during my QBasic programming era. But at some point, once I knew C coding, I decided to come back to my old DOS programming environment and try to rotate a cube. I didn't want to use any assembler routines, so what I did was to draw the cube in wireframe and then fill the inside with the regular fill() instruction. It worked out quite ok. The nicest trick is probably that I used not the regular 13h mode (320x200x256 colors) but the mode 7 (320x200, 16 colors). That allowed me to have fast swapping routine, and also to do some nice trick. For example, as long as my 3d objects didn't have more than 16 triangles (or faces), I could assign one fixed color to each of them, and then do the shading by changing the appropriate entry in the palette table and simulate 256 colors. Neat!



The code is bellow, I don't even want to have a look to it... Just... Enjoy if you can. I know it works, cause it still runs here in my XP machine.

```
DEFINT A-Z
TYPE CARA
    c1 AS INTEGER
    c2 AS INTEGER
    c3 AS INTEGER
    c4 AS INTEGER
END TYPE

TYPE PUN2D
    x AS INTEGER
    y AS INTEGER
```

```
END TYPE
```

```
TYPE PUN3D
```

```
    x AS SINGLE
```

```
    y AS SINGLE
```

```
    z AS SINGLE
```

```
END TYPE
```

```
DECLARE FUNCTION ProyectaX (p AS PUN3D)
```

```
DECLARE FUNCTION ProyectaY (p AS PUN3D)
```

```
DECLARE SUB Pixel3D (p AS PUN3D, Col%)
```

```
DECLARE SUB Linea3D (a AS PUN3D, b AS PUN3D, Col%)
```

```
DECLARE SUB Traslada (p AS PUN3D, vect AS PUN3D)
```

```
DECLARE SUB Rota (p AS PUN3D, p AS PUN3D, aX AS SINGLE, aY AS SINGLE, aZ AS SINGLE)
```

```
DECLARE SUB DibujaPoly (n AS INTEGER, Col%)
```

```
DECLARE SUB PoneColor (Col%, r%, v%, a%)
```

```
SCREEN 7
```

```
DIM SHARED Posicion AS PUN3D
```

```
DIM SHARED Vertice(0 TO 7) AS PUN3D
```

```
DIM SHARED VScreen(0 TO 7) AS PUN2D
```

```
DIM SHARED Poly(0 TO 5) AS CARA
```

```
DIM SHARED Col(0 TO 5) AS INTEGER
```

```
DIM SHARED Medio(0 TO 5) AS PUN3D
```

```
DIM SHARED normal(0 TO 5) AS PUN3D
```

```
DIM SHARED AuxVertice(0 TO 7) AS PUN3D
```

```
DIM SHARED AuxMedio(0 TO 5) AS PUN3D
```

```
DIM SHARED AuxNormal(0 TO 5) AS PUN3D
```

```
DIM SHARED Orden(0 TO 5) AS INTEGER
```

```
Poly(0).c1 = 3
```

```
Poly(0).c2 = 2
```

```
Poly(0).c3 = 1
```

```
Poly(0).c4 = 0
```

```
Poly(1).c1 = 4
```

```
Poly(1).c2 = 5
```

```
Poly(1).c3 = 6
```

```
Poly(1).c4 = 7
```

```
Poly(2).c1 = 0
```

```
Poly(2).c2 = 1
```

```
Poly(2).c3 = 5
```

```
Poly(2).c4 = 4
```

```
Poly(3).c1 = 1
```

```
Poly(3).c2 = 2
```

```
Poly(3).c3 = 6
```

```
Poly(3).c4 = 5
Poly(4).c1 = 2
Poly(4).c2 = 3
Poly(4).c3 = 7
Poly(4).c4 = 6
Poly(5).c1 = 3
Poly(5).c2 = 0
Poly(5).c3 = 4
Poly(5).c4 = 7
```

```
Col(0) = 1
Col(1) = 2
Col(2) = 3
Col(3) = 4
Col(4) = 5
Col(5) = 6
```

```
'cube
Vertice(0).x = -20
Vertice(0).Y = -20
Vertice(0).z = -20
Vertice(1).x = 20
Vertice(1).Y = -20
Vertice(1).z = -20
Vertice(2).x = 20
Vertice(2).Y = -20
Vertice(2).z = 20
Vertice(3).x = -20
Vertice(3).Y = -20
Vertice(3).z = 20
Vertice(4).x = -20
Vertice(4).Y = 20
Vertice(4).z = -20
Vertice(5).x = 20
Vertice(5).Y = 20
Vertice(5).z = -20
Vertice(6).x = 20
Vertice(6).Y = 20
Vertice(6).z = 20
Vertice(7).x = -20
Vertice(7).Y = 20
Vertice(7).z = 20
```

```
FOR i% = 0 TO 5
    DIM p1 AS PUN3D
    DIM p2 AS PUN3D
    DIM p3 AS PUN3D
    DIM p4 AS PUN3D
```

```
DIM u AS PUN3D
DIM v AS PUN3D
DIM n AS PUN3D

p1 = Vertice(Poly(i%).c1)
p2 = Vertice(Poly(i%).c2)
p3 = Vertice(Poly(i%).c3)
p4 = Vertice(Poly(i%).c4)

mx = (p1.x + p2.x + p3.x + p4.x) / 4
my = (p1.y + p2.y + p3.y + p4.y) / 4
mz = (p1.z + p2.z + p3.z + p4.z) / 4

Medio(i%).x = mx
Medio(i%).y = my
Medio(i%).z = mz

u.x = p2.x - p1.x
u.y = p2.y - p1.y
u.z = p2.z - p1.z

v.x = p3.x - p1.x
v.y = p3.y - p1.y
v.z = p3.z - p1.z

n.x = u.y * v.z - u.z * v.y
n.y = v.x * u.z - u.x * v.z
n.z = u.x * v.y - u.y * v.x

mo! = SQR(n.x * n.x + n.y * n.y + n.z * n.z)
normal(i%).x = n.x / mo!
normal(i%).y = n.y / mo!
normal(i%).z = n.z / mo!

NEXT i%

Posicion.x = 0
Posicion.y = 0
Posicion.z = 100

aX! = 0
aY! = 0
aZ! = 0

FOR i% = 0 TO 5
    Orden(i%) = i%
NEXT i%

DO
```

```
aX! = aX! + .01
aY! = aY! + .07
aZ! = aZ! + .05

FOR i% = 0 TO 7
    Rota Vertice(i%), AuxVertice(i%), aX!, aY!, aZ!
    Traslada AuxVertice(i%), Posicion
    d! = 200 / AuxVertice(i%).z
    VScreen(i%).x = 160 + AuxVertice(i%).x * d!
    VScreen(i%).y = 100 - AuxVertice(i%).y * d!
NEXT i%

FOR i% = 0 TO 5
    Rota Medio(i%), AuxMedio(i%), aX!, aY!, aZ!
    Traslada AuxMedio(i%), Posicion
    Rota normal(i%), AuxNormal(i%), aX!, aY!, aZ!
NEXT i%

FOR i% = 0 TO 5
    FOR j% = 0 TO i% - 1
        z1 = AuxMedio(Orden(i%)).z
        z2 = AuxMedio(Orden(j%)).z

        IF (z1 > z2) THEN
            o% = Orden(i%)
            Orden(i%) = Orden(j%)
            Orden(j%) = o%
        END IF
    NEXT j%
NEXT i%

SCREEN , , 1, 0
CLS
FOR i% = 0 TO 5

    'IF (AuxMedio(Orden(i%)).z < Posicion.z) THEN
    '    DibujaPoly Orden(i%), Col(Orden(i%))
    'END IF

    IF (AuxNormal(Orden(i%)).z > 0) THEN
        DibujaPoly Orden(i%), Col(Orden(i%))
    END IF

NEXT i%
PCOPY 1, 0

LOOP WHILE INKEY$ = ""
```

```

DEFSNG A-Z
SUB DibujaPoly (n AS INTEGER, Col%)
    DIM p1 AS PUN2D
    DIM p2 AS PUN2D
    DIM p3 AS PUN2D
    DIM p4 AS PUN2D

    p1 = VScreen(Poly(n).c1)
    p2 = VScreen(Poly(n).c2)
    p3 = VScreen(Poly(n).c3)
    p4 = VScreen(Poly(n).c4)

    mx = (p1.x + p2.x + p3.x + p4.x) / 4
    my = (p1.y + p2.y + p3.y + p4.y) / 4

    r% = (63 * AuxNormal(n).z)
    CALL PoneColor(Col%, r%, .5 * r%, .5 * r%)

    LINE (p1.x, p1.y)-(p2.x, p2.y), Col%
    LINE (p2.x, p2.y)-(p3.x, p3.y), Col%
    LINE (p3.x, p3.y)-(p4.x, p4.y), Col%
    LINE (p4.x, p4.y)-(p1.x, p1.y), Col%
    PAINT (mx, my), Col%, Col%
END SUB

SUB Linea3D (a AS PUN3D, b AS PUN3D, Col%)
    x1 = ProyectaX(a)
    y1 = ProyectaY(a)
    x2 = ProyectaX(b)
    y2 = ProyectaY(b)

    LINE (x1, y1)-(x2, y2), Col%
END SUB

SUB Pixel3D (p AS PUN3D, Col%)
    PSET (ProyectaX(p), ProyectaY(p)), Col%
END SUB

SUB PoneColor (Col%, r%, v%, a%)

    OUT &H3C8, Col%
    OUT &H3C9, r%
    OUT &H3C9, v%
    OUT &H3C9, a%

END SUB

DEFINT A-Z
FUNCTION ProyectaX (p AS PUN3D)
    ProyectaX = 200 * p.x / p.z

```

```
END FUNCTION

FUNCTION ProyectaY (p AS PUN3D)
    ProyectaY = 200 * p.Y / p.z
END FUNCTION

DEFSNG A-Z
SUB Rota (ori AS PUN3D, dest AS PUN3D, aX AS SINGLE, aY AS SINGLE, aZ AS SINGLE)
    ' eje Y
    xa = COS(aY) * ori.x - SIN(aY) * ori.z
    za = SIN(aY) * ori.x + COS(aY) * ori.z
    ' eje Z
    dest.x = COS(aZ) * xa + SIN(aZ) * ori.Y
    ya = COS(aZ) * ori.Y - SIN(aZ) * xa
    ' eje X
    dest.z = COS(aX) * za - SIN(aX) * ya
    dest.Y = SIN(aX) * za + COS(aX) * ya
END SUB

SUB Traslada (p AS PUN3D, vect AS PUN3D)
    p.x = p.x + vect.x
    p.Y = p.Y + vect.Y
    p.z = p.z + vect.z
END SUB
```

Enjoy your nostalgia moment :)

[inigo quilez](https://iquilezles.org/articles/qbasiccube/) - learning computer graphics since 1994